

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL
INTELLIGENCE
AND EXPERT SYSTEMS

COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I Nikhil Kumar Reddy Kadiveti (192425031) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	Medical Diagnosis Expert System: Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	Crop Disease Advisory System: Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	Automobile Fault Diagnosis System: Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	Student Academic Advisory System: Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	Cybersecurity Threat Diagnosis System: Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

Stage 1 — Problem Analysis and Domain Understanding

1.1 Problem Definition

A vehicle may show symptoms that point to different mechanical or electrical faults. A rule-based expert system can capture a bounded set of diagnostic relationships and derive probable faults from observed evidence. The system is designed as an educational diagnostic assistant: it does not replace a mechanic or physical inspection.

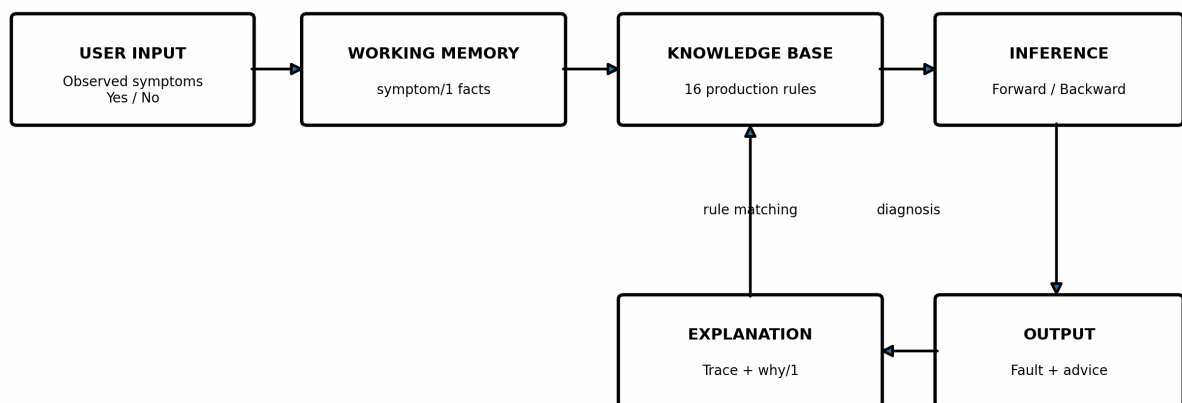
1.2 Domain Entities

- Input facts: observable vehicle symptoms such as starting problems, engine noises, warning indicators, vibration, smoke, mileage changes and brake noise.
- Goals: probable faults such as weak battery, faulty starter motor, low engine oil, worn engine mounts and fuel-pump failure.
- Rules: relationships connecting a complete set of symptoms to a fault and recommended action.
- Working memory: dynamic symptom/1 and known_fault/1 facts.

1.3 Domain Model

User symptoms → Working Memory → Production Rule Knowledge Base → Inference Engine → Diagnosed Fault(s) → Explanation + Recommended Action.

Automobile Fault Diagnosis Expert System — Architecture



Explanation facility makes each conclusion traceable.

Stage 2 — Knowledge Acquisition and Representation

Rubric target: relevant domain knowledge should be acquired and represented accurately using well-structured Prolog facts and rules. filecite turn1file0 L146-L173

2.1 Knowledge Acquisition Approach

The diagnostic relationships are organized from standard automotive diagnostic concepts covering starting, battery/charging, ignition, lubrication, exhaust, vibration, cooling and braking. For the academic knowledge source record, the project uses the Robert Bosch GmbH Automotive Handbook as a general automotive reference and the official SWI-Prolog documentation for implementation details.

The 11th edition of the Bosch Automotive Handbook was published by Robert Bosch GmbH in 2022 and is a comprehensive automotive engineering reference. cite turn1search0 turn1search7

2.2 Knowledge Specification

Knowledge Statement	Prolog Representation
Slow crank + dim dashboard + battery warning → weak battery	rule(weak_battery,[slow_crank,dashboard_dim_lights,warning_light_battery],Advice)
Clicking sound + no start → faulty starter	rule(faulty_starter_motor,[clicking_sound,engine_wont_start],Advice)

Abnormal driving vibration → wheel misalignment	rule(wheel_misalignment,[abnormal_vibration_driving],Advice)
Squealing brakes → worn brake pads	rule(worn_brake_pads,[squealing_brakes],Advice)

2.3 Representation Principle

Each rule stores three components: the conclusion/fault, the list of required conditions, and a human-readable recommendation. This keeps the knowledge base modular and allows the same rules to support both inference directions.

Stage 3 — Production Rule / Knowledge Base Design

Rubric target: a comprehensive, consistent and logically structured production-rule knowledge base.
filecite turn1file0 L174-L206

3.1 Rule Structure

```
rule(
    weak_battery,
    [slow_crank, dashboard_dim_lights, warning_light_battery],
    'Battery is weak or discharged. Charge or replace the battery.'
).
```

3.2 Complete Rule Set

#	IF conditions	THEN fault
1	slow_crank + dashboard_dim_lights + warning_light_battery	weak_battery
2	clicking_sound + engine_wont_start	faulty_starter_motor
3	engine_wont_start + rough_idle + reduced_mileage	faulty_spark_plug
4	rough_idle + black_smoke_exhaust + reduced_mileage	clogged_air_filter
5	dashboard_dim_lights + warning_light_battery + stalling	faulty_alternator
6	engine_noise_ticking + engine_wont_start	worn_timing_belt
7	engine_noise_knocking + rough_idle + stalling	faulty_ignition_coil
8	warning_light_oil + engine_noise_knocking	low_engine_oil
9	engine_noise_knocking + reduced_mileage + fuel_smell	exhaust_leak
10	abnormal_vibration_idle + engine_noise_knocking	worn_engine_mounts
11	abnormal_vibration_driving	wheel_misalignment
12	abnormal_vibration_driving + reduced_mileage	low_tire_pressure
13	black_smoke_exhaust + reduced_mileage + warning_light_engine	faulty_oxygen_sensor
14	overheating + warning_light_engine	radiator_overheating_issue
15	squealing_brakes	worn_brake_pads
16	engine_wont_start + stalling + fuel_smell	fuel_pump_failure

3.3 Quality Checks

- All 16 rules have non-empty condition lists.
- No two rules use the same complete condition set.
- Shared symptoms are intentional, allowing multiple supported faults to be derived.
- Every conclusion has an associated advice string.

- The knowledge base separates domain knowledge from the inference procedure.

Stage 4 — Prolog Implementation and Programming

Rubric target: fully functional Prolog implementation using facts, predicates, rules, variables, unification and backtracking. filecite turn1file0 L207-L235

4.1 Program Architecture

File	Responsibility
knowledge_base.pl	18 symptoms, 16 rules, working memory, forward chaining, backward chaining, explanation
auto_expert.pl	Four automated demonstration scenarios
auto_expert_interactive.pl	User-facing yes/no console and goal verification
procedural_equivalent.py	Imperative comparison for Stage 7
test_run_log.txt	Recorded expected demonstration results
README.md	Setup, commands, project structure and scope
diagrams/	Architecture, inference and UI visuals

4.2 Prolog Features Demonstrated

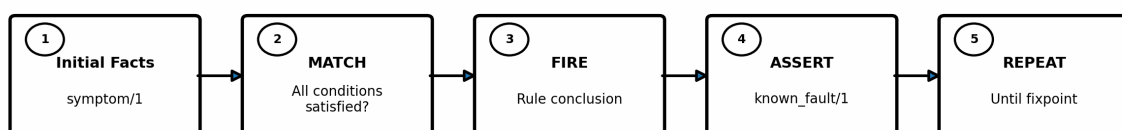
- Facts: known_symptom/1 and rule/3.
- Variables: Fault, Conditions and Advice in rule queries.
- Dynamic predicates: symptom/1 and known_fault/1.
- Recursion: condition-list traversal and forward-chain repetition.
- Unification: matching a requested fault against rule/3.
- Backtracking: alternative rule clauses can be explored by Prolog.
- Database control: assertz/1 and retractall/1 manage working memory.

SWI-Prolog documents dynamic predicates as predicates whose clauses can be changed during execution with assert/retract operations, and specifies dynamic/1 for such predicates. cite turn0search4 turn0search3

Stage 5 — Forward Chaining Implementation and Demonstration

The excellent band requires correct data-driven reasoning from initial facts through applicable rules to final conclusions. filecite turn1file0 L238-L266

Forward Chaining — Data-Driven Reasoning



Example: slow_crank + dashboard_dim_lights + battery_warning
→ weak_battery → add conclusion → scan again

5.1 Algorithm

```
forward_chain :-
    forward_loop,
    print_faults.
```

```
forward_loop :-
    rule(Fault, Conditions, Advice),
    \+ known_fault(Fault),
```

```

all_symptoms_present(Conditions),
!,
assertz(known_fault(Fault)),
format('Rule fired -> ~w~n', [Fault]),
forward_loop.
forward_loop.

```

5.2 Demonstration

Initial facts:
slow_crank, dashboard_dim_lights, warning_light_battery

MATCH:
all three conditions are present

FIRE:
weak_battery

ASSERT:
known_fault(weak_battery)

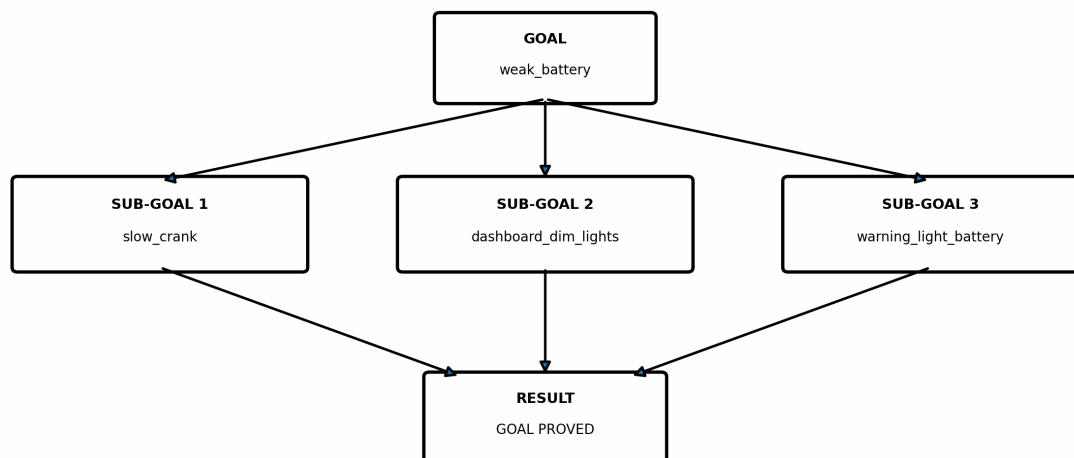
REPEAT:
no additional applicable rule

RESULT:
weak_battery

Stage 6 — Backward Chaining Implementation and Demonstration

The excellent band requires effective goal-driven reasoning using Prolog's inference mechanism and a clear trace. filecite turn1file0 L267-L296

Backward Chaining — Goal-Driven Reasoning



Every required condition must be proved; one missing condition causes failure.

6.1 Goal-Proving Logic

```

prove(Goal) :-
    rule(Goal, Conditions, _Advice),
    prove_all(Conditions).

```

```

prove_all([]).
prove_all([S|Rest]) :-
    symptom(S),
    prove_all(Rest).

```

6.2 Successful Goal

Goal: faulty_starter_motor
Required: clicking_sound, engine_wont_start

[OK] clicking_sound
[OK] engine_wont_start

GOAL PROVED

6.3 Failed Goal

Goal: fuel_pump_failure

Required: engine_wont_start, stalling, fuel_smell

[MISSING] engine_wont_start

GOAL FAILED: insufficient evidence

6.4 Explanation Facility

The why/1 predicate reports the selected goal, the rule conditions and the recommended action, making the reasoning traceable instead of returning only a fault name.

Stage 7 — Procedural and Non-Procedural Paradigm

Analysis

The rubric expects a clear distinction plus practical demonstration in the Prolog system.

filecite turn1file0 L297-L326

Aspect	Prolog / Declarative	Python / Procedural
Main idea	State facts and relationships	Specify instructions
Rule addition	Add rule/3 clause	Add explicit if block
Matching	Unification and rule queries	Manual set comparison
Search	Prolog resolution/backtracking	Explicit loops/flags
Inference direction	Same knowledge base supports both	Separate procedures are clearer
Working memory	Dynamic facts	Mutable Python data structures

7.1 Practical Python Contrast

```
if {"slow_crank", "dashboard_dim_lights",  
    "warning_light_battery"} <= symptoms:  
    faults.append(("weak_battery",  
                  "Battery is weak or discharged."))
```

The Python code must explicitly encode the check. In Prolog, the relationship is represented as a rule and the inference engine performs the matching.

Stage 8 — Inference, Unification and Reasoning Accuracy

The excellent band requires accurate conclusions using unification, backtracking and logical inference across diverse cases. filecite turn1file0 L327-L352

8.1 Unification

A query such as rule(weak_battery, Conditions, Advice) is unified with the matching rule clause. Prolog binds Conditions and Advice to the values stored in that clause.

8.2 Backtracking

If a goal or rule path fails, Prolog can reconsider alternative clauses and continue the search. This is different from a hand-written loop because the search mechanism is part of the logic-programming environment.

8.3 Accuracy and Coverage

Evaluation Dimension	Evidence
Rule coverage	16 production rules cover starting, electrical, engine, fuel, cooling, vibration and braking symptoms.
Multiple-fault reasoning	Test Case 3 derives both low_engine_oil and worn_engine_mounts.
Negative reasoning	Fuel-pump goal fails when engine_wont_start is absent.
Consistency	No duplicate complete condition sets in the designed rule base.
Traceability	Forward rule-fire traces + backward [OK]/[MISSING] traces + why/1 explanation.

The supplied test log therefore provides evidence for positive diagnosis, multiple-fault diagnosis and controlled failure. The project should not claim clinical/industrial-level diagnostic accuracy because the knowledge base is intentionally bounded.

Stage 9 — Testing, Results and System Demonstration

The excellent band calls for comprehensive test cases, accurate outputs, reasoning traces and a convincing live demonstration. filecite turn1file0 L355-L380

Case	Symptoms	Forward Result	Backward Result
1	slow_crank + dashboard_dim_lights + warning_light_battery	weak_battery	PROVED
2	clicking_sound + engine_wont_start	faulty_starter_motor	PROVED
3	abnormal_vibration_idle + engine_noise_knocking + warning_light_oil	low_engine_oil + worn_engine_mounts	BOTH PROVED
4	squealing_brakes	worn_brake_pads	fuel_pump_failure FAILED

9.1 Important Interpretation of Test Case 4

The supplied batch log labels Case 4 as a no-matching-fault scenario, but the actual output shows that forward chaining fires worn_brake_pads from squealing_brakes. The negative result applies to the backward goal fuel_pump_failure, which correctly fails because engine_wont_start is missing. This report explicitly records that distinction.

Stage 10 — Documentation, GitHub and Technical Presentation

10.1 Final Repository Structure

```

automobile-fault-diagnosis-expert-system/
├── knowledge_base.pl
├── auto_expert.pl
├── auto_expert_interactive.pl
├── procedural_equivalent.py
├── test_run_log.txt
├── README.md
├── .gitignore
├── diagrams/
│   ├── architecture.png
│   ├── forward_chaining.png
│   ├── backward_chaining.png
│   └── interactive_demo.png

```

10.2 GitHub URL

<https://github.com/NikhilKumarReddyKadiveti/MLA0104-AIES/tree/main/CO5%20AT1>

10.3 Technical Presentation / Viva Checklist

- Explain the domain and why a rule-based expert system is appropriate.
- Explain one production rule using IF–THEN terminology.
- Demonstrate forward chaining from symptoms to a fault.
- Demonstrate backward chaining from a chosen fault to required symptoms.
- Explain unification, dynamic facts and backtracking.
- Show the multiple-fault case.
- Show the insufficient-evidence case.
- Open the GitHub repository and explain each file.

Conclusion and Evaluation

The completed system is intentionally aligned to the ten-stage assessment format. It contains a structured automotive knowledge base, a reusable Prolog inference engine, both forward and backward reasoning, an explanation facility, an interactive user interface, procedural comparison code, multiple test scenarios and professional documentation assets. The design directly addresses the assessment's emphasis on production rules, logical inference, declarative/non-procedural programming and reasoning accuracy.

Limitations and Future Enhancements

- Expand the knowledge base with more vehicle-specific rules and confidence levels.
- Add weighted evidence or certainty factors for ambiguous symptoms.

- Store vehicle make/model/year as additional facts.
- Add a graphical interface or web interface.
- Add automated regression tests inside SWI-Prolog.
- Validate individual diagnostic rules against workshop/service data before real-world use.

References

1. SWI-Prolog Documentation. Official documentation and reference material. <https://www.swi-prolog.org/Documentation.html> cite turn0search2
2. SWI-Prolog Reference Manual. Documentation of dynamic predicates and runtime clause manipulation. cite turn0search24 turn0search4
3. Robert Bosch GmbH, Automotive Handbook, 11th Edition, John Wiley & Sons, 2022, ISBN 978-1-119-91190-6. cite turn1search0 turn1search7

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.
5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts	7–8% – Forward chaining works correctly with minor	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.

		through applicable rules to final conclusions.	limitations.		
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				